LOCUSCARCERUM

A static memory manager with two boundaries in one word, proved in Lean 4

BY

DOUGLAS QUIGG

RESEARCH AND THEORY
MEMORY MANAGEMENT

MMGR
2026

© 2026 Douglas Quigg
AGPL-3.0-or-later OR LicenseRef-Commercial OR LicenseRef-Educational

ABSTRACT

An embedded program has no system allocator, and its memory manager can be smaller for that reason. We present the cellblock of MMgr: a pool fixed at compile time, with a persistent tier that grows up from the base and a temporary tier that grows down from the size. The two boundaries are the two halves of one atomic word. A take is one fetch-add on that word, which moves its own boundary and reads the other in one atomic step. A take that lands past the other boundary adds its bytes to an overrun that the next writer settles. A give is one store and one fetch-or. Writers hold the cellblock alone behind a drain of the turnstiles of embedded_types.

The Lean 4 model proves the invariant of the word through every interleaving of a round, that a refused take is final, and that two words or an undo break it. It proves the layout of both tiers, the spill, the trim and the owner word. CBMC, symbolic execution of the machine code at `-O0` and `-O2`, and GenMC under five memory models prove the C against the model. The manager is 778 bytes of Xtensa at `-Os`. Four axioms are named.

CONTENTS

1 LOCUSCARCERUM 1
1.1 1. Introduction 1
1.2 2. The static pool and the warden 2
2.1 Pools 2
2.2 The warden 3
2.3 Owners 4
1.3 3. The algorithm of the two boundaries 4
3.1 The bounds word 4
3.2 Cells and the two tiers 5
3.3 The bump, the overrun and the settle 6
3.4 The give, the trim bits and the boundary 7
3.5 Passes, the writer and the drain 9
3.6 The owner word and the movable option 10
1.4 4. Formal verification 12
4.1 Packed against Split: Lemmas 1 to 6 12
4.2 The layout, the take, the spill and the readings 13
4.3 The owner word and the ticket ring 14
4.4 Translation validation: the C against the Lean 14
1.5 5. Empirical evaluation 15
5.1 The suite and the libc oracle 15
5.2 Size 16
5.3 Fuzz, stress and the trim race 16
5.4 The A/B bench 17
1.6 6. Related work 17
1.7 7. Limitations and conclusion 18

BIBLIOGRAPHY 21

1

LOCUSCARCERUM

This paper describes and proves the cellblock of MMgr, a memory manager for embedded C11 programs ISO C11 [1]. A cellblock is a pool of bytes fixed at compile time, divided into a persistent tier that grows up from the base and a temporary tier that grows down from the size. The two boundaries are the two halves of one atomic word, and a take in either tier is one fetch-add on that word inside a pass through the turnstiles of embedded_types [3]. The implementation is `src/locus_carcerum/locus_carcerum.c` and its header, over the pool macros of `include/mmgr.h`. The model is the Lean 4 project under `theory/locus_carcerum` Lean 4 [2], and the proof of the C against the Lean is the harness `test/harness.py prove`: CBMC on the source CBMC [4], symbolic execution of each toolchain's machine code angr [5], and GenMC on the thread clients under five memory models GenMC [6]. Every claim below is one of four kinds, and the paper names the kind each time: a theorem that Lean checks, a bound that CBMC or the machine arm exhausts, an execution space that GenMC exhausts under a named memory model, or an axiom that the artifact takes on trust from the target, the host or the caller.

1.1 1. INTRODUCTION

A general allocator is one object with many invariants. An allocator with size classes, such as jemalloc jemalloc [12] or mimalloc mimalloc [13], keeps free lists per class, per thread and per page, and its correctness is the correctness of the coupling between them. A Two-Level Segregated Fit allocator, TLSF TLSF [14], bounds the time of a take by a bit map over size classes and pays for it in headers and in the bit map. In both the proof of the allocator, where one exists, is the proof of the whole: a block is free because the list that holds it says so, and the list is right because every path that frees a block also fixes the list.

We take a narrower position, and the position is the one the embedded target forces. The storage is static. A program declares each pool with its size as a constant, and the linker places the bytes. No cellblock ever grows, moves or returns storage to a system allocator, because there is no system allocator to return it to. Under that constraint the manager needs two things and no more: a boundary per tier, and a way to give a cell back. The boundary is one half of one word. The give is one store to the cell's header. Everything else, the compaction of a tier, the trim of a boundary over freed cells, the growth of a cell in place, is the work of one writer that holds the cellblock alone, and the writer holds it by a drain of the turnstiles and not by a lock.

The design has one claim about its algorithm, and the claim is about the word. A take moves its own boundary by one fetch-add and reads the other boundary in the same atomic step, because

the two boundaries are the two halves of one word. The take that lands past the other boundary does not undo its move. It adds its bytes to an overrun counter, and the next writer settles the boundary back by that amount. We prove that under this protocol a bump refused in a round stays refused for that round (Lemma 2), that the accepted takes of a round are contiguous (Lemma 3), and that the state invariant stands through every interleaving of takes and gives between two writers (Theorem 1). We also prove the two negatives that justify the design: an undo of a refused bump breaks the invariant (Section 4.1), and two words in place of one admit an interleaving where an earlier reserve is refused and a later one accepted above it (Lemma 6). The negatives are theorems in Lean, not remarks.

The paper is organized as follows. Section 2 is the static pool and the warden, the compile-time surface of the manager. Section 3 is the algorithm of the two boundaries as the C has it: the word, the two tiers, the bump and the settle, the give and the trim bits, the pass and the drain, and the owner word. Section 4 is the Lean 4 model and the proof of the C against it. Section 5 is the empirical evaluation. Section 6 is related work, and Section 7 names the trust boundary and ends.

1.2 2. THE STATIC POOL AND THE WARDEN

2.1 Pools

A pool is a region of bytes with a name, and `include/mmgr.h` gives a program three ways to declare one. `ParsMemoriaeInternae(name_, bytes_)` places `bytes_` bytes in the program's image, aligned to `MMGR_ALIGN_BYTES`. `ParsMemoriaeExternum(name_, bytes_)` places the same array under `MMGR_EXTRAM_ATTR`, the attribute a port supplies for its external memory, and exists only where `MMGR_ENABLE_EXTRAM` holds. `ParsMemoriaeHospes(name_)` names a pool whose bytes the host gives at run time, with no storage and no size. The internal form is:

```
#define ParsMemoriaeInternae(name_, bytes_)
    MMGR_PARS_DECLARED_ONCE(name_);
    MMGR_PARS_TOKEN(name_);
    enum { name_##_bytes = (bytes_) };
    EMBED_ALIGN(MMGR_ALIGN_BYTES) static uint8_t mmgr_pars_storage_##name_[bytes_];
    static const uint8_t *const name_ EMBED_UNUSED = mmgr_pars_storage_##name_;
    EMBED_STATIC_ASSERT(sizeof(mmgr_pars_storage_##name_) == name_##_bytes, ...)
```

Two of the lines are guards that fail the build or the link in place of a check at run time. `MMGR_PARS_DECLARED_ONCE` is an `enum` value whose name carries the pool's; a second declaration of the same name in one translation unit collides on it. `MMGR_PARS_TOKEN` is a global `int` with external linkage; a second translation unit that declares the same pool fails the link on the symbol defined twice. A third guard, `MMGR_PARS_CLAIMED_ONCE`, is the same `enum` device for the cellblock that dresses the pool, so that one pool is under one cellblock and no more. These three guards are the whole of the defense the manager has against a pool used twice, and none of them runs.

2.2 The warden

A cellblock is a pool under a guard. `LocusCarcerum(prisonsite_, ...)` declares a site and every cellblock built at it, from a list of `MMGR_MINIMUM_SECURITY(name_)` and `MMGR_MAXIMUM_SECURITY(name_)` items:

```
#define MMGR_MINIMUM_SECURITY(name_) (MinimumSecurityGuard, name_, 0)
#define MMGR_MAXIMUM_SECURITY(name_) (MaximumSecurityGuard, name_, 1)

#define LocusCarcerum(prisonsite_, ...)
    MMGR_CARCER_WALK(MMGR_CARCER_BODY, prisonsite_, __VA_ARGS__)
    EMBED_TABLE_STORAGE struct
    {
        MMGR_CARCER_WALK(MMGR_CARCER_MEM, prisonsite_, __VA_ARGS__)
    } prisonsite_ EMBED_UNUSED = {MMGR_CARCER_WALK(MMGR_CARCER_SEAT, prisonsite_, __VA_ARGS__)}
```

The walk `MMGR_CARCER_WALK` expands one reader over every item, and `EMBED_NARG` picks the walk with the right count of items, `MMGR_CARCER_W1` to `MMGR_CARCER_W8`, so a site holds up to eight cellblocks. Three readers run. `MMGR_CARCER_BODY` emits a `static SeriesCellularum` for the cellblock, initialized by `MMGR_CARCER_STATE(base, bytes)`, and one static entry per guard member that forwards to the module's function with that cellblock's state. `MMGR_CARCER_MEM` emits the member of the site struct, typed by the guard. `MMGR_CARCER_SEAT` emits the initializer of that member, the table of the entries. Everything the macro emits is data with its initial value, and a cellblock's first byte is the address of its own pool, which the linker resolves. Nothing runs at startup.

The site is the warden: a `const` struct of guards, one per cellblock, and every call reaches a cellblock through it. A program with `LocusCarcerum(ram, MMGR_MINIMUM_SECURITY(general))` takes with `ram.general.persistent_buf_alloc(size, owner)` and gives with `ram.general.persistent_buf_release(p, owner)`. The two guard types have the same thirteen members and differ in what their release and growth do to the bytes:

```
typedef struct
{
    void *(*persistent_buf_alloc)(size_t size, const CellOwner *owner);
    void (*persistent_buf_release)(void *prisoner, const CellOwner *owner);
    void *(*temporary_buf_alloc)(size_t size, const CellOwner *owner);
    embed_bool (*who_owns_buf)(const void *at);
    size_t (*buf_available)(void);
    void *(*buf_grow)(void *prisoner, size_t size, const CellOwner *owner);
    embed_bool (*movable_init)(const CarcerMovableInitCfg *args);
    const CarcerMovableEntry *(*movable_buf_alloc)(size_t size, const CellOwner *owner);
    embed_bool (*movable_buf_release)(const CarcerMovableEntry *cell, const CellOwner *owner);
    embed_bool (*movable_buf_read)(const CarcerMovableCfg *args);
    embed_bool (*movable_buf_write)(const CarcerMovableCfg *args);
    embed_iword (*movable_buf_compare)(const CarcerMovableCfg *args);
    size_t (*movable_buf_measure)(const CarcerMovableCfg *args);
} MinimumSecurityGuard;
```

A `MaximumSecurityGuard` has the same members. Its release zeroes the payload before the give, and its growth zeroes the bytes it adds. A minimum security cellblock has no release that zeroes

and a maximum security one has no plain release, so the choice is made once at the declaration and no call can make the other one. The bytes, the state and the warden are all static, so a declaration in a header gives each translation unit that includes it a site of its own.

`MMGR_CARCER_BODY` carries two static assertions per cellblock: the pool holds at least one cell, `2 * MMGR_CARCER_ALIGN` bytes, and the pool is at most `MMGR_CARCER_BYTES_MOST`, the most bytes a half of the boundary word can name (Section 3.1). A pool that fails either does not build.

2.3 Owners

Every take and every give names an owner, and the owner is a row of one table that the program declares once with `MMGR_OWNERS`:

```
typedef struct
{
    const char *name;
    size_t place;
    embed_atomic_usize in_use;
    embed_atomic_usize hw;
} CellOwner;
```

`place` is the index of the row, from 1, and 0 names no owner. `in_use` is the payload bytes of the cells the owner holds now, and `hw` the highest value `in_use` has reached. A take that names an owner outside the table, or `NULL`, is refused before it reaches the word. `MMGR_CARCER_OWNERS`, 128 by default, sizes the place field of the owner word (Section 3.6), and a static assertion holds the table's length under it. The owner is the unit of accounting and of the undo: the cellblock knows which owner holds which bytes, and the undo of one owner's work (Section 4.3) gives back that owner's cells and no other's.

1.3 3. THE ALGORITHM OF THE TWO BOUNDARIES

3.1 The bounds word

A cellblock's state is one struct, and its turnstiles are its last member:

```
typedef struct
{
    uint8_t *const base;
    const size_t size;
    embed_atomic_usize bounds;
    embed_atomic_usize persistent_overrun;
    embed_atomic_usize temporary_overrun;
    embed_atomic_usize in_use;
    embed_atomic_usize persistent_in_use;
    embed_atomic_uptr movable;
    embed_atomic_word closed;
    embed_atomic_usize sink;
    EmbedTurnstiles turnstiles;
} SeriesCellularum;
```

`bounds` carries both boundaries. With H = `MMGR_CARCER_HALF_BITS`, half the bits of `embed_atomic_usize_value`, the persistent end E is the low half and the temporary top T the high half:

$$W = T \cdot 2^H + E, \qquad E = W \bmod 2^H, \qquad T = \lfloor W/2^H \rfloor.$$

At rest over a pool of S bytes the word is $S \cdot 2^H$: the end at 0, the top at S. The C reads the halves with one mask and one shift:

```
#define MMGR_CARCER_HALF_BITS ((unsigned int)((sizeof(embed_atomic_usize_value) * 8u) / 2u))
#define MMGR_CARCER_HALF_MASK \
    ((embed_atomic_usize_value)(((embed_atomic_usize_value)1u << MMGR_CARCER_HALF_BITS) - 1u))
#define MMGR_CARCER_BYTES_MOST ((size_t)1u << (MMGR_CARCER_HALF_BITS - 1u))
#define MMGR_CARCER_WANT_MOST ((size_t)1u << (MMGR_CARCER_HALF_BITS - 8u))
```

Two limits keep a move inside its half. A pool is at most `MMGR_CARCER_BYTES_MOST`, 2^{H-1} bytes, and one bump asks at most `MMGR_CARCER_WANT_MOST`, 2^{H-8} bytes of payload. A bump up adds its need n to W, which adds n to E and carries into T only where $E + n \geq 2^H$. A bump down subtracts $n \cdot 2^H$ from W, which subtracts n from T and borrows out of the word, never into E. With L bumps in flight, each of need at most $m = h + 2^{H-8}$ for header h, the end reads at most 2^{H-1} before the read-modify-write of any bump, and the bumps in flight move it by at most Lm afterwards. The static assertion in the C is

$$S + L\,(h + 2^{H-8}) < 2^H,$$

with L the lanes of the turnstiles, and it holds for 63 lanes at $H = 32$ and for 31 lanes at $H = 16$. The Lean file `Word.lean` states these as `bumpUp_no_carry`, `bumpDown_borrow_leaves`, `in_flight_within`, `caps_64`, `caps_32`, `lanes_64` and `lanes_32`. A bump down that takes the top below 0 wraps it modulo 2^H to a value above $2^H - Lm > S$, and the fit test after the bump refuses it; that is `bumpDown_wrap_refused` and `bumpDown_in_flight_within`.

The request guard is in the same file. The C rounds a request up to a multiple of `MMGR_CARCER_ALIGN`, and a request near the top of the word would round to a small cell. Every take and every growth refuses a request above the cellblock's size before it rounds, so the round and the header added to it stay below the word: `round_wraps_unguarded` exhibits the wrap, and `round_guarded`, `need_guarded`, `guard_64` and `guard_32` state the guard.

3.2 Cells and the two tiers

A cell is a header word, a payload and an owner word. `CARCER_WORD` is `sizeof(embed_atomic_usize)` rounded up to `MMGR_CARCER_ALIGN`, and `CARCER_HDR` is two of them, the header and the owner word together:

```
#define CARCER_FREE  ((embed_atomic_usize_value)0u)
#define CARCER_BUSY  ((embed_atomic_usize_value)1u)
#define CARCER_STATE ((embed_atomic_usize_value)3u)
#define CARCER_WORD \
    ((sizeof(embed_atomic_usize) + (MMGR_CARCER_ALIGN - 1u)) & ~(MMGR_CARCER_ALIGN - 1u))
#define CARCER_HDR (2u * CARCER_WORD)
```

The header word is the payload size in its high bits and the cell's state in its two low bits, `CARCER_FREE` or `CARCER_BUSY`. The owner word is Section 3.6. A payload is a multiple of `MMGR_CARCER_ALIGN`, so the low bits of the size are free for the state. The persistent tier is its cells laid end to end from offset 0, and its end e_P is the sum of their sizes with their headers. The temporary tier is the same cells laid from the top t_Q up to S. In the formal document the header size is $h = 16$ and a cell is a pair (o, p) that occupies $[o, o + h + p)$; the C answers the take with $o + 8$, the first payload byte.

`Layout.lean` is the persistent tier as a list of cells with `foot`, `len`, `busyCells` and `count`, the payload each owner holds. `Temporary.lean` is the temporary tier as the persistent tier mirrored: the same cells read in the same order, with the base at the cellblock's size and the boundary at the start of the first cell. Every operation of the persistent tier is an operation of the temporary tier with that one change, and the file states it as such. The sweep that joins free runs is `Layout.merge` for both; the trim that returns the free run at the boundary is `Layout.trimEnd` for the persistent tier and `trimStart` for the temporary one; the fit that lays a take over a free run is `Spill.firstFit` from offset 0 for the persistent tier and `lastFit` before the size for the temporary tier; and `laidHigh`, which lays a temporary take at the high end of its run, is `Take.laid` reversed, `laidHigh_reverse`.

3.3 The bump, the overrun and the settle

A take is a bump. `carcer_bumped` is the whole of the fast path, and its one atomic read-modify-write is the fetch-add on `bounds`:

```
const embed_atomic_usize_value ahead = embed_atomic_usize_load(&series->bounds);
const size_t within = carcer_all(take->want <= MMGR_CARCER_WANT_MOST) &
                      (down | carcer_all(carcer_bounds_end(ahead) <= MMGR_CARCER_BYTES_MOST));
const size_t allowed = permit & take->permit & within;
const size_t need = (CARCER_HDR + take->want) & allowed;
const embed_atomic_usize_value moved =
    (embed_atomic_usize_value)carcer_pick(down, (0u - need) << MMGR_CARCER_HALF_BITS, need);
const embed_atomic_usize_value word = embed_atomic_usize_fetch_add(&series->bounds, moved);
const size_t end_was = carcer_bounds_end(word);
const size_t top_was = carcer_bounds_top(word);
const size_t was = carcer_pick(down, top_was, end_was);
const size_t across = carcer_pick(down, end_was, top_was);
const size_t now = carcer_pick(down, top_was - need, end_was + need);
const size_t fits_up =
    carcer_all(now <= across) & carcer_all(now >= was) & carcer_all(across <= series->size);
const size_t fits_down =
    carcer_all(now >= across) & carcer_all(now <= was) & carcer_all(now <= series->size);
const size_t taken = allowed & ((fits_up & up) | (fits_down & down));
(void)embed_atomic_usize_fetch_add(overrun, need & ~taken);
```

`carcer_all` is every bit set for a true condition and none for a false one, and `carcer_pick` selects by such a mask, so the path has no branch on the data. In the formal document a bump up of need $n = h + p$ is

$$(w, T_w) := (E, T), \quad W := W + n, \qquad \tau \equiv (w + n \le T_w) \wedge (w + n \ge w) \wedge (T_w \le S),$$

$$\tau \Rightarrow P \;:= P \cdot (w, p),\ \text{answer } w + 8, \qquad \neg\tau \Rightarrow O_E \;:= O_E + n,\ \text{answer } \bot,$$

and a bump down is the mirror on T with O_T. The first line is the one fetch-add; w and T_w are the two halves of the old value, read in the same atomic step as the add. The refused branch does not subtract n back. It adds n to the tier's overrun, `persistent_overrun` or `temporary_overrun`, and the boundary stays where the bump left it. A bump whose want is above `MMGR_CARCER_WANT_MOST`, or whose half already reads above `MMGR_CARCER_BYTES_MOST`, has `within` clear: its need is 0, its fetch-add adds 0, and it goes to the path of the writer with the word not moved.

The state in the document is $\sigma = (E, T, O_E, O_T, P, Q)$ with P a chain from 0 to e_P and Q a chain from t_Q to S, and the invariant is

$$I(\sigma) \equiv E = e_P + O_E \ \wedge\ T = t_Q - O_T \ \wedge\ e_P \le t_Q.$$

Rest is $R(\sigma) \equiv I(\sigma) \wedge O_E = 0 \wedge O_T = 0$. The settle is the first step of every writer, with every pass out of the turnstiles:

$$E \;:= E - O_E, \quad O_E \;:= 0, \quad T \;:= T + O_T, \quad O_T \;:= 0.$$

```
EMBED_INLINE void carcer_settled(SeriesCellularum *series)
{
    const embed_atomic_usize_value word = embed_atomic_usize_load(&series->bounds);
    const size_t end = carcer_bounds_end(word);
    const size_t top = carcer_bounds_top(word);
    const size_t over_end = (size_t)embed_atomic_usize_load(&series->persistent_overrun);
    const size_t over_top = (size_t)embed_atomic_usize_load(&series->temporary_overrun);

    embed_atomic_usize_store(&series->bounds, carcer_bounds(end - over_end, top + over_top));
    embed_atomic_usize_store(&series->persistent_overrun, 0u);
    embed_atomic_usize_store(&series->temporary_overrun, 0u);
}
```

The stores are plain because the writer is alone (Section 3.5). A round is the interval between two writers, and inside a round only bumps and gives run. Section 4.1 proves that I stands through a round and that the settle restores R.

A bump the gap refuses does not fail. The take closes the cellblock, runs the drain of the turnstiles, settles, and walks its tier alone: `carcer_spilled` sweeps the tier, which joins every run of free cells into one (`carcer_swept`), lays the take over the first free run that holds it (`carcer_split`), and where none does and the gap has room, bumps at the boundary. Only where the sweep finds no run and the gap has no room does the take return `NULL`. The sweep and the split are `Layout.sweep`, `Take.laid` and `Spill.spilled` in Lean, and no path in the C branches on what another context did.

3.4 The give, the trim bits and the boundary

A give is one store to the cell's header, and then a question: does the cell stand at its tier's boundary? `carcer_given` is:

```
mmgr_zero_buf(safe, cell.size & zeroing & cell.proceeds);
carcer_foot_claimed(series, cell.off, cell.size, persistent, cell.proceeds);
embed_atomic_usize_store(carcer_word_picked(series, cell.proceeds, cell.off),
                         carcer_word(cell.size, CARCER_FREE));
EMBED_ATOMIC_ORDERED();
const size_t closed = carcer_all((embed_atomic_word_load(&series->closed) & CARCER_CLOSED) != 0u);
const embed_atomic_usize_value word = embed_atomic_usize_load(&series->bounds);
const size_t at_boundary =
    closed | carcer_pick(persistent,
                         carcer_all(carcer_end_of(cell.off, cell.size) == carcer_bounds_end(word)),
                         carcer_all(cell.off == carcer_bounds_top(word)));
```

The zero fill is `size & zeroing & proceeds` bytes: the whole payload where the guard zeroes and the give proceeds, and nothing otherwise (`Entry.zeroed`). The claim of the owner word, `carcer_foot_claimed`, counts the cell's bytes off its owner once (Section 3.6). The store sets the header free. A full fence follows, and only then does the give read the closed bit and the boundaries. A cell at the boundary, or any cell while the closed bit holds, goes on to the trim.

The trim is where the gives meet the writers, and the meeting is on the closed word. The word has four bits:

bit	name	meaning
0	`CARCER_CLOSED`	a writer holds the cellblock
1	`CARCER_CLOSED_SLEEPERS`	a pass sleeps on the word with the futex contract
2	`CARCER_TRIM_PERSISTENT`	a give left the persistent end owed a trim
3	`CARCER_TRIM_TEMPORARY`	a give left the temporary top owed a trim

A give at the boundary sets the closed bit and its tier's trim bit with one fetch-or. Where the old value had the closed bit clear the give now holds the cellblock, and it trims: it walks from the boundary one header at a time over free cells, stops at the first busy one or at the end, and stores the place it stopped as the new boundary. Where another context held the bit the give leaves, and the trim bit stays owed on the word. Every writer opens in one pass, `carcer_opened`:

```
EMBED_INLINE void carcer_opened(SeriesCellularum *series)
{
    static void (*const wakes[2])(embed_atomic_word *) = {carcer_wake_none, embed_word_wake};
    const embed_atomic_word_value taken =
        embed_atomic_word_fetch_and(&series->closed, ~CARCER_TRIM_PENDING);

    EMBED_ATOMIC_ORDERED();
    carcer_owed_trimmed(series, taken & CARCER_TRIM_PENDING);
    const embed_atomic_word_value was =
        embed_atomic_word_fetch_and(&series->closed, ~(CARCER_CLOSED | CARCER_CLOSED_SLEEPERS));
    wakes[(was >> 1u) & 1u](&series->closed);
}
```

The first fetch-and takes the trim bits and the context that holds the bit trims the tiers they name. The second clears the closed bit and the sleeper bit whatever trim bits stand, so no stream of gives holds a writer back. A trim bit a give sets between the two stays on the open word, and

the next take or the next read of the gap, which starts with one load of the word (`carcer_owed_serviced`), holds the bit and runs the trim before its own pass. The consequence is the one the take needs: a take reads a boundary with no free cell at it, so the gap it reads is the gap there is.

`Boundary.lean` is one tier's cells by place, with the trim as a walk that reads one header at a time and stores the boundary after it stops. `Trim.lean` is the closed word as the writers and the gives share it, for any number of contexts, with a tier `dirty` while a cell given back at its boundary stands with no trim run over it. The theorems are `Boundary.open_trimmed`, `Boundary.owed_walked`, `Trim.open_trimmed` and `Trim.taken_trimmed`: a cellblock with no trim bit set and no give in flight has no free cell at either boundary, and the take that the context with the bit runs first leaves no tier dirty except through a give in flight. `Trim.lean` also checks one interleaving of a protocol without trim bits, where a free cell stays at the boundary.

3.5 Passes, the writer and the drain

Every operation of Section 3.3 and Section 3.4 runs inside a pass, and a pass is an entry into the turnstiles of embedded_types [3]. The turnstiles are a manifold of lanes, each one atomic word on its own cache line. A pass enters with one fetch-or on a lane, leaves with one fetch-add, and a writer runs its drain by one read of each open lane and a wait on the busy ones. `carcer_attempted` is the pass:

```
const unsigned int at =
    embed_turnstile_enter(&series->turnstiles, (embed_atomic_uptr_value)(uintptr_t)missed);
const size_t closed = carcer_all((embed_atomic_word_load(&series->closed) & CARCER_CLOSED) != 0u);
const size_t permit = ~closed;
...
const uintptr_t answer = work(series, context, permit);

embed_turnstile_leave(&series->turnstiles, at);
*missed = ~permit;
```

The pass enters at the address of the caller's own word, reads the closed bit, and runs its work with `permit` every bit set where the cellblock is open and none where it is closed. Closed, the stores of the work land on `sink`, a word of the state that nothing reads, and each fetch-add it makes adds 0: the work runs the same instructions either way and touches nothing of the cellblock. A pass that missed backs off and enters again, `carcer_passed`:

```
const embed_atomic_word_value read = embed_atomic_word_load(&series->closed);
embed_turnstile_backed_off(&series->closed, read, CARCER_CLOSED_SLEEPERS, &backoff,
                           (unsigned int)((read & CARCER_CLOSED) != 0u));
answer = carcer_attempted(series, id, work, context, &missed);
```

The backoff is a pause inside its first unit and then a sleep on the closed word that doubles to its ceiling, with the sleeper bit set by one fetch-or before the sleep. The last argument gates the sleep: a pass sleeps only where the value it read holds the closed bit, because the open of a writer wakes only a word it closed, and a value read open has no writer to wake it.

The writer is `carcer_shut`:

```
EMBED_INLINE void carcer_shut(SeriesCellularum *series)
{
    EmbedTurnstileBackoff backoff;
    embed_atomic_word_value was = embed_atomic_word_fetch_or(&series->closed, CARCER_CLOSED);

    embed_turnstile_backoff_begin(&backoff);
    while ((was & CARCER_CLOSED) != 0u)
    {
        embed_turnstile_backed_off(&series->closed, was, CARCER_CLOSED_SLEEPERS, &backoff, 1u);
        was = embed_atomic_word_fetch_or(&series->closed, CARCER_CLOSED);
    }
    EMBED_ATOMIC_ORDERED();
    embed_turnstiles_drain(&series->turnstiles);
    carcer_settled(series);
}
```

The writer takes the closed bit with a fetch-or, backs off on the word while another writer holds it, issues a sequentially consistent fence, runs the drain, and settles. Every pass that entered before the fence and still holds a lane is below the open count the drain reads, and the drain waits for each such pass to leave. Every pass that enters after the fence reads the closed bit set. The writer therefore works alone, and every store of Section 3.3 and Section 3.4 that is plain is plain for that reason.

Four Lean files carry this. `Pass.lean` is the passes and the writer as whole steps, and `alone_exclusive` states that while a writer works alone no pass works on the cellblock. Its `drained` step takes every pass idle as its hypothesis, and `drained_lane` discharges that hypothesis from `Manifold.Drain.drain_lane_complete` of embedded_types. `Order.lean` carries the whole steps over to C11. Every write to the closed word is a read-modify-write, so C11 gives them one modification order, and the two sequentially consistent fences of a pass and a drain stand in the total order S of fences. `pass_lane_below_open` and `drain_ends_pass` carry the two theorems of the turnstile, and `drain_or_closed` is the disjunction the drain needs: the pass reads the closed bit set, or the drain reads the lane of the pass busy and waits it out. `unfenced_admits` exhibits the execution, with the fences removed, that violates the disjunction, and `give_or_walk` is the same disjunction for a give and the walk of a writer: the give reads the closed bit set, or the walk reads the header free. `Sleep.lean` is the sleeper bit for one writer and one pass: `wait_bounded` carries `Manifold.Backoff.limit_fits`, `gated_woken` proves that a gated sleeper is never asleep with the writer at rest, and `ungated_sleeps` exhibits the sleep without the gate that no wake ends. `Futex.lean` is the host's address wait against the model wait that GenMC runs (Section 4.4): `real_in_model` and `model_in_real` state that the two reach the same states, `whole_in_low` that a model on the 32-bit compare of Linux holds every host, and `low_sleeps` that a model on the whole word would miss a sleep that Linux takes. `one_wake_loses` checks that a wake of one waiter leaves a second asleep, which is why the C wakes all.

3.6 The owner word and the movable option

The owner word stands behind the payload. From the high bit down its fields are the owner's place in `MMGR_OWNER_PLACE_BITS` bits, one completion bit, three signal bits, and a stamp of the clock in the bits left:

```
#define MMGR_OWNER_SIGNAL_BITS 3u
#define MMGR_OWNER_TIME_BITS (EMBED_WORD_BITS - MMGR_OWNER_PLACE_BITS - 1u - MMGR_OWNER_SIGNAL_BITS)
#define MMGR_OWNER_TIME_MASK (((embed_atomic_usize_value)1u << MMGR_OWNER_TIME_BITS) - 1u)
#define MMGR_OWNER_COMPLETE \
    ((embed_atomic_usize_value)1u << (MMGR_OWNER_TIME_BITS + MMGR_OWNER_SIGNAL_BITS))
#define MMGR_OWNER_PLACE_SHIFT (MMGR_OWNER_TIME_BITS + MMGR_OWNER_SIGNAL_BITS + 1u)
#define MMGR_OWNER_PLACE_MASK \
    ((((embed_atomic_usize_value)1u << MMGR_OWNER_PLACE_BITS) - 1u) << MMGR_OWNER_PLACE_SHIFT)
```

`MMGR_OWNER_PLACE_BITS` is the fewest bits every place fits, from 4 to 16, chosen from `MMGR_CARCER_OWNERS` at compile time. 1024 owners on a 64-bit word leave 49 bits of the stamp; on a 32-bit word they leave 17. A take writes the word as `carcer_owner_word`: the place shifted into its field, or the low bits of `mmgr_carcer_hw_clock_ns()`, a weak function whose default returns 0 and which a port defines to its clock.

The completion bit is the device by which a cell comes off its owner once. A give that names the cell's owner, and an undo that passes the cell, each claim the word with one fetch-or of `MMGR_OWNER_COMPLETE`; the context whose fetch-or found the bit clear counts the bytes off, and every other claim counts nothing:

```
const embed_atomic_usize_value was = embed_atomic_usize_fetch_or(
    carcer_word_picked(series, permit, carcer_foot_of(off, size)), MMGR_OWNER_COMPLETE & permit);
const size_t first = permit & carcer_all((was & MMGR_OWNER_COMPLETE) == 0u) &
                     carcer_all(carcer_owner_place(was) != 0u);
```

A give that names another owner has `permit` clear: its fetch-or sets nothing and it counts nothing. `Give.lean` states this over any sequence of claims on one cell, `claims_once`: the bytes counted off are the cell's size where at least one claim had a permit, and none where none had one.

The movable option is a table of entries outside the pool, given once through `movable_init`, and a movable cell is one the mover may move:

```
struct CarcerMovableEntry
{
    embed_atomic_usize word;
};
```

An entry's word is where its cell stands, and a caller holds the entry and not the address, so a move is one store to the word. A movable take claims an entry with one fetch-or on a bit of the table's claim words, and the take whose fetch-or found the bit clear names the entry; `Entry.one_name` states that no two contexts name one entry. The mover is a writer. It reads the cellblock quiet, runs the drain, lays each busy cell over the first free run below it the same way a take does (`mmgr_movable_buf_move`), and `Take.move_count` states that the running count of each owner stays the count the layout holds, so the release, which counts off what the header says, takes the owner back to what it was. A read or a write through an entry is a span, and `Bounds.fits_inside` states that a span the test passes lies inside its cell without ever forming `offset + bytes`, which can wrap.

1.4 4. FORMAL VERIFICATION

The Lean project is `theory/locus_carcerum`, nineteen files under `LocusCarcerum/`, and its lakefile requires `EmbeddedTypes`, the project of the turnstile paper. It builds with Lean `v4.35.0`, and the proofs workflow fails where any theorem closes with `sorry`. Bit-level statements are on `BitVec` at 64 and 32 bits and close with `bv_decide` or `decide`; arithmetic closes with `omega`; transition systems close by induction over an inductive `Reach`. This section states the theorems as the files state them, and then the proof of the C against them.

4.1 Packed against Split: Lemmas 1 to 6

`Packed.lean` is the state σ of Section 3.3 with a bump as one step: the read-modify-write on its own boundary and its read of the other boundary are one transition. That is what one word with both boundaries is. Its definitions are `State`, `Inv`, `Rest`, `initial`, `bumpUp`, `bumpDown`, `settle` and `Round`, a round as any sequence of bumps, and its theorems are the lemmas of the document.

Lemma 1 (monotone in a round). In a round E does not decrease and T does not increase. A bump up adds $n > 0$ to E, a bump down subtracts $n > 0$ from T, and a give changes neither.

Lemma 2 (a refused bump is final). In a round, where a bump up B_1 is refused, every bump up B_2 whose read-modify-write follows the read-modify-write of B_1 is refused. The proof is one step: $w_2 \geq w_1 + n_1 > T_1 \geq T_2$. The inequality $T_1 \geq T_2$ holds because B_2 reads T in the same atomic step as its read-modify-write, and that step follows the step of B_1. The Lean theorems are `crossed_refuses_up` and `crossed_refuses_down`.

Lemma 3 (accepted bumps are contiguous). Where $B_1, \ldots, B_k$ are the accepted bumps up of a round in read-modify-write order, $o_1 = e_P$ at the start of the round and $o_{i+1} = o_i + n_i$. By Lemma 2 every accepted bump precedes every refused one, so nothing moves E between two accepted bumps. The theorems are `taken_up_at_end` and `taken_down_at_start`.

Lemma 4 (I stands through a round). Where $R(\sigma)$ holds at the start of a round, $I(\sigma)$ holds after the two atomic operations of every bump. E rises by every need, O_E by every refused need, and e_P by every accepted need, so $E - O_E = e_P$; an accepted bump up has $w + n \leq T \leq t_Q$, so $e_P \leq t_Q$. The theorems are `inv_bumpUp`, `inv_bumpDown` and `inv_round`.

Lemma 5 (the settle restores rest). Where $I(\sigma)$ holds and no bump is in flight, the settle gives $R(\sigma)$: `rest_settle` and `settle_ends`.

Theorem 1. From σ_0, under any interleaving of bumps and gives between writers, every writer finds $R(\sigma)$ after its settle, and every returned address lies in a cell of $P \cup Q$ that overlaps no other. The Lean statement is `round_then_settle`:

```
theorem round_then_settle (s s' : State) (hR : Rest s) (h : Round s s') :
    Rest (settle s') ∧ s'.eP ≤ s'.tQ
```

The two negatives are theorems too. Section 6.1 of the document replaces the refused branch with $E := E - n$ and exhibits six bumps after which a bump X is accepted at $o = 6112$ while $e_P = 6088$: the chain has a gap and the cell of X lies past E. That is why the overrun exists and the undo does not.

Lemma 6 (two words are not final). `Split.lean` is the same state with a bump as two steps, `reserveUp` and `decideUp`, `reserveDown` and `decideDown`, as two atomic words have it. The reserve

is a read-modify-write on the boundary of the bump itself and the decision a later read of the other boundary, and steps of different bumps interleave. The theorem `not_absorbing` exhibits the interleaving: from rest at $E =$ `fd4c0` and $T =$ `fd868` with $S = 2^{20}$, a bump A of need `1e8` reserves, a bump B of need `a8` reserves above it and decides, a bump down C of need `210` moves T to `fd658`, and then A decides and is refused. A reserved before B and is refused after B is accepted; after the round $E - O_E =$ `fd568` $< e_P =$ `fd750`, and `not_inv_after` states that the invariant is false. `settle_below_laid` states the consequence: the settle puts the persistent end below a laid cell, and the next bump up is accepted over it.

```
theorem not_absorbing :
    let (wA, s1) := reserveUp 0x1e8 before
    let (wB, s2) := reserveUp 0xa8 s1
    let (_, s4) := reserveDown 0x210 (decideUp wB 0xa8 s2)
    wA < wB ∧ (wB + 0xa8 ≤ s2.T) ∧ ¬ (wA + 0x1e8 ≤ s4.T)
```

Lemma 6 is the design decision. Two boundaries in two words is the natural layout, and it is wrong; the one fact that makes Lemma 2 true, $T_1 \geq T_2$, is the fact that a bump reads the other boundary in the step that moves its own.

4.2 The layout, the take, the spill and the readings

`Layout.lean` proves the sweep over the persistent tier: `merge_len`, `merge_busy`, `merge_count` and `merge_head_busy` state that the join of free runs keeps every byte laid, the busy cells in order and every owner's count, and `merge_separated` that no two free cells stand together after it. `trimEnd_ends`, `trimEnd_busy` and `trimEnd_len` state that the trim leaves no free cell at the end, keeps the busy cells and the counts, and never raises the end. `sweep_busy`, `sweep_count`, `sweep_len` and `sweep_ends` compose them.

`Take.lean` proves `carcer_split`: `laid_len` that the run's bytes stay laid, `carried_least` and `carried_most` that the cell carries at least the request and less than the request, a header and the alignment more, and `laid_count` that the owner's count rises by what the cell carries. `move_count` is the mover (Section 3.6). `Spill.lean` proves `firstFit_spec`, `spill_len`, `spill_count`, `bump_len` and `bump_count` for the spill and the bump at the end, and `grownUp_len`, `grownUp_count` and `grownDown_owner_word` for the two growths: a persistent cell grows in place and a temporary cell moves down with its owner word where it stood. `Temporary.lean` proves the mirror of each: `trimStart_*`, `sweepDown_*`, `lastFit_spec`, `lastFit_nearest`, `laidHigh_len`, `laidHigh_count`, `spillDown_len`, `spillDown_count`, `bumpDown_len` and `bumpDown_count`.

`Fragment.lean` is the reading `mmgr_series_cellularum_fragmentation`: the bytes the persistent tier occupies that no payload holds, against the hundredth of the bytes it occupies, rounded up. The divisor is a hundredth rather than the holes times a hundred, because that product can pass what a narrow word counts. `reading_le` states that with the holes at most the bytes occupied the reading is at most 100. `Bounds.lean` proves `fits_inside` and `refused_outside` for the span test, `owns_iff` for `mmgr_who_owns_buf`, and `raise_ge_count` and `raises_ge` for the high-water marks. `Entry.lean` proves `one_name` by induction over `Reach` and `zeroed_whole` and `zeroed_none` for the zero fill.

4.3 The owner word and the ticket ring

`OwnerWord.lean` is the word of Section 3.6 on `BitVec`. `fields_64` and `fields_32` state, at 1024 owners and ten place bits, that the four fields do not overlap and fill the word:

```
theorem fields_64 :
    placeMask64 &&& complete64 = 0 ∧ placeMask64 &&& signal64 = 0 ∧
      placeMask64 &&& sequenceMask64 = 0 ∧ complete64 &&& signal64 = 0 ∧
      complete64 &&& sequenceMask64 = 0 ∧ signal64 &&& sequenceMask64 = 0 ∧
      (placeMask64 ||| complete64 ||| signal64 ||| sequenceMask64) = ~~~0
```

`word_fields_64` and `word_fields_32` state that a word built from a place and a sequence gives both back. The file reads the low field as a sequence, one count per take of the owner, and defines `after c mark` as a distance modulo 2^b above 0 and below 2^{b-1}. `after_exact` and `before_exact` state that the reading is exact while fewer than 2^{b-1} takes come between the mark and the undo. The undo of one owner's work back to a mark, `undo`, gives back the owner's cells after the mark and keeps each other cell: `undo_keeps_others`, `undo_keeps_before` and `undo_removes`. `undos_commute` states that the undo of one owner and the undo of another give the same cells in either order, which is what makes the undo a function of the cells and of no schedule.

`Ticket.lean` states the cellblock as an instance of the ticket ring that embedded_types [3] proves. The completion fetch-or is `Flags.transition` with the completion bit as the flag: `complete_is_transition`, `complete_once`, and `complete_keeps_place`, `complete_keeps_sequence`, `complete_keeps_signal`. A give is a claim on one slot, `give_counts_as_claims`, and `give_bytes_once` is then `Claim.claims_at_most_one`:

```
theorem give_bytes_once (size s : Nat) (gives : List (Bool × Nat)) :
    Give.claims size false (gives.map Prod.fst) ≤ size
```

The owner's sequence is a ticket: `sequence_window_distinct` is `TicketRing.window_distinct`, two takes fewer than 2^b apart have sequences of their own, and `sequence_roll` is `TicketRing.place_roll`. The claim bit of a movable entry is a push, `entry_push` and `entry_push_once`, and `release_drained` is `Claim.claim_drained`: the release of a movable cell runs inside a pass, and the drain of a move waits for that pass.

4.4 Translation validation: the C against the Lean

The Lean proves the model. `test/harness.py prove` proves the C against the model, and against its own machine code, in three arms that `test/prove/prove.py` runs, which is translation validation [10] of each artifact.

CBMC on the source. Three harnesses under `test/prove` each state one Lean file's theorem as an assertion over every sequence of steps up to a bound, and CBMC unwinds the C with every source the library links to that bound, with an assertion on each unwind, so a loop that runs past the bound is a failure and not a gap:

harness	Lean files	bound
`locus_carcerum_fragmentation_prove.c`	`Fragment`	no loop
`locus_carcerum_sequence_prove.c`	`Layout`, `Take`, `Spill`, `Give`, `Temporary`	`PROVE_STEPS=2`, unwind 12
`locus_carcerum_movable_prove.c`	`Take`, `Bounds`, `Entry`	`PROVE_STEPS=2`, unwind 12

The sequence harness runs every sequence of `PROVE_STEPS` operations on a small cellblock, each a take of either tier at any of four sizes or a give of any cell in place, plain or with a zero fill, and asserts after each step that every cell in place keeps its bytes and counts to its owner and that the gap is the top less the end; once every cell goes back, both boundaries are where they started and every count is 0. The movable harness adds the movable take, the read and the write at any offset and length, the move and the release, and asserts that every cell keeps its bytes through every move and that no two cells share an entry. The movable harness needs more than 27 GB under CBMC, and a hosted runner has 16 GB; `prove --hosted` names it and does not run it, and it runs on a workstation.

Symbolic execution of the machine code. The same three harnesses compile with `gcc` and with `clang` at `-O0` and at `-O2`, and `test/prove/symbolic.py` runs each binary under angr 10.0.5 [5], every path to its end, and answers `proved` where no feasible path reaches `prove_failed`. This arm is what makes the proof a proof of the artifact and not of the source: a compiler that changes the order of the fetch-add and the fit test, or that folds the mask arithmetic of `carcer_all` into a branch, fails here and nowhere else.

GenMC on the thread clients. Two clients state the concurrent theorems, and GenMC v0.20.0 exhausts every execution of each under five memory models, `--sc`, `--tso`, `--rc11`, `--imm` and `--arm8` RC11 [7] IMM [8] ARMv8 [9], with `--unroll=8`, `--disable-spin-assume`, `--disable-load-annotation` and `--disable-loop-jump-threading` so that the loops of the backoff and the drain are run and not assumed. `locus_carcerum_trim_genmc.c` is two threads that give the two cells at the temporary top at one instant; the assertion is that once both join the top stands at the cellblock's end and nothing remains owed, which is `Trim.open_trimmed` over the C. `locus_carcerum_writers_genmc.c` is two writers on one closed bit, a temporary take only a spill can meet against the give of the cell at the top; whichever holds the bit first, the other backs off and sleeps on the closed word or leaves its trim owed, and once both join the cellblock is idle, nothing owed and every owner at zero. `genmc_host.h` gives GenMC the address wait as a model, and `Futex.lean` proves that model sound against the host contract (Section 3.5): a model execution blocked asleep is one a real waiter can sleep forever in, and no other.

1.5 5. EMPIRICAL EVALUATION

5.1 The suite and the libc oracle

The test suite is 276 CTest targets over every compile-time width, at 100% of lines, branches and functions, and it runs green against this library and against libc. `python test/harness.py ab` builds two trees one after the other, `build` as the library ships and `build-oracle` with every entry that has a libc equivalent replaced by that equivalent, and runs the suite in both; the A/B agrees

where both trees pass. The cellblock has no libc equivalent, so its tests run the same in both trees, and what the oracle build checks for it is that no test of a module above it depends on a behavior libc does not have. Six trees in all carry the suite: the two above, `build-cov` instrumented with `always_inline` and link time optimization off, two DMA trees with recovery and the boundary word check on and off, and `build-extram` with external memory on.

5.2 Size

`.text` per translation unit, compiled alone with no LTO and no link, on Xtensa for the esp32s3 with gcc 14.2:

translation unit	-O0	-O1	-Os	-O2	-O3
`locus_carcerum`	3703	870	778	834	878

The whole manager, both tiers, the spill, the trim, the growth, the owner accounting, the movable option and the turnstile pass, is 778 bytes at `-Os`. The library as a whole against newlib on a cortex-m4 at `-Os`, for the same set of jobs, is 16428 bytes against 38104, and the comparison is `tools/dev_env/against_libc.py`.

5.3 Fuzz, stress and the trim race

`test/fuzz/locus_carcerum_fuzz.c` is a libFuzzer harness over every entry of the guard on one cellblock: takes of both tiers at any size, a size past the cellblock's among them; gives plain and with a zero fill; growths plain and with a zero fill; and gives and growths that name the wrong owner. Every operation runs against a model of the cells in place, every payload is filled with its cell's mark and read back before it goes, and the owner reads, the gap, the fragmentation reading, the round and the zero fill run beside them. A second cellblock with the movable option runs movable takes, writes, reads, compares, measures, gives and moves against a copy of each cell's bytes. Each cellblock must read idle once every cell goes back. `python test/harness.py fuzz` builds the harness with Clang and runs it in the suite, and `--seconds 60 --filter locus_carcerum` runs it over a corpus that grows between runs.

`test/stress/locus_carcerum_stress.c` is the stress program, `locus_carcerum_stress_host`, and its arguments are threads, rounds, cellblock bytes, runs and seed:

```
build/test/host/locus_carcerum_stress_host 16 200000 1048576 1 777
```

Each thread takes and gives on an owner of its own. The cellblock's storage and its state each lie between pages that fault on any touch, so a write that leaves its cell or its word faults at once and does not land on a neighbor. Every payload holds the mark of its thread and reads back before its give, and the cellblock must read idle once every thread ends. `test/stress/locus_carcerum_trim_race.c` is two threads that give the two cells at the temporary top at one instant, round after round; whichever give comes second finds the other's trim begun or done, every round must end with the top wound back over both, and a round that ends with a free cell at the top counts as lost and fails the program. The race program is the stress form of `Trim.open_trimmed`, as the GenMC client is the form that runs every execution.

5.4 The A/B bench

`test/bench/bench_carcer_ab.c` times the cellblock against the arena of a project beside it, ProtoCore, on the same host with the same cycle counter: `persist alloc+free` at 32 and 256 bytes, `transient run+release` over sixteen temporary takes, `zero` against `wipe`, `max security alloc+zeroing release` against `secure take+wiped release`, `min security alloc+release` against `plain take+release`, and `span walk`. It prints one CSV row per case, the implementation, the case, the bytes, the operations per microsecond and the bytes per microsecond, and the build leaves it out where no ProtoCore checkout stands beside the repository. The bench measures; it is not a test, and no number from it is a claim of this paper.

1.6 6. RELATED WORK

jemalloc jemalloc [12] and **mimalloc** mimalloc [13] are each a general allocator over a system allocator. jemalloc keeps an arena per thread and size classes per arena, with runs of pages per class; mimalloc splits the free list per page and keeps a local free list and two lists for frees from other threads per page, so that a free from another thread is one atomic push. Both grow and return memory, both carry data about their classes in proportion to them, and neither has a proof. The cellblock has one class, the cell, and two boundaries; it never grows and never returns; and the price is that a take the gap cannot meet walks the tier alone. The point of comparison is not speed but what has to be true for the manager to be right, and for the cellblock that is one word and one invariant.

TLSF TLSF [14] is the allocator of real time systems: Two-Level Segregated Fit, with a bit map over size classes, so that a take and a give are $O(1)$ with a bounded constant. The cellblock's take is also $O(1)$, one fetch-add, and so is its give, one store and one fetch-or, but the bound has a different shape: a take the gap cannot meet is $O(\text{cells})$, the sweep of its tier, and it runs alone. TLSF pays for its bound in a header per block and a bit map per level; the cellblock pays in the gap, which is the one place a persistent and a temporary take can both land. Fragmentation is the measure both care about Wilson [15], and the cellblock's reading of it is a theorem (`Fragment.reading_le`), not an estimate.

seL4 seL4 [16] is a verified operating system with a proof from the abstract specification to the binary, and its memory management is the division of raw memory into objects through capabilities. The cellblock makes no such claim: its proofs are of an algorithm and its C, not of a system. What it has in common with seL4 is the proof of the binary, Section 4.4, which for seL4 is a translation of the binary into the logic and for the cellblock is a symbolic execution of the machine code of each toolchain to the end of every path.

CompCert CompCert [17] proves the compiler and so makes the proof of the source a proof of the binary. The cellblock takes the other route and checks each compiled artifact, because the targets it builds for, gcc and clang for Xtensa, ARM and RISC-V, are not CompCert. **Iris** Iris [18] is a concurrent separation logic in which a concurrent data structure is proved against a sequential specification, each operation as if it ran alone at one instant. The cellblock's proof is in that spirit without the logic: the pass is the atomic step, the drain is what makes it one, and the turnstile theorems that make the drain sound come from the project that proves them [3]. **CakeML** CakeML [19] is a verified implementation of a language down to machine code, with a verified

collector of unused memory. The cellblock has no collector; its gives are explicit, and what it proves about them is that each cell comes off its owner once and that a trim leaves no free cell at a boundary.

1.7 7. LIMITATIONS AND CONCLUSION

The trust boundary is four axioms, and the paper has named each where it is used.

- **Axiom A1, the atomics of the target.** Each `embed_atomic_*` operation is one atomic step with the memory order of its arm, and `EMBED_ATOMIC_ORDERED` is a sequentially consistent fence as C11 states it. The Lean model takes each operation as one step. GenMC checks the two clients under five models of the hardware, and the machine arm checks each toolchain's reordering of the sequential harnesses, but neither checks the hardware itself.
- **Axiom A2, the wait of the host.** The address wait compares and sleeps as one step with respect to the wake, and a wake ends every sleep on the word. `Futex.lean` proves the GenMC model sound against this contract, and `Sleep.lean` proves the gate sound under it. A host whose wait is a timed sleep keeps safety and loses the bound on the sleep.
- **Axiom A3, the pass that leaves.** A drain returns when every pass of its fence has left, and not before. A pass that never leaves holds its lane and the writer waits without end. That is the contract a caller signs at the turnstile, and the cellblock does not detect a context that dies with a lane.
- **Axiom A4, the owner table.** `MMGR_OWNERS` is declared once, every owner a program names is in it, and `MMGR_CARCER_OWNERS` bounds its length. A take that names an owner outside the table is refused, but a program that declares two tables fails only at the link.

Two facts about the owner word are stated as the C has them. The low field is a stamp of the clock from `mmgr_carcer_hw_clock_ns`, whose weak default returns 0, and `OwnerWord.lean` reads that field as a sequence that a counter gives one take at a time. The undo theorems therefore hold for a port whose clock is monotone and whose takes of one owner fall more than one tick apart, and the window of `after_exact` is 2^{b-1} ticks of that clock; a port that supplies no clock writes 0 into every take, and the undo is then the undo of every cell of the owner. The three signal bits are the cellblock's to raise and read with one atomic step each, and no theorem here is about them.

The bounds of Section 4.4 are bounds. CBMC proves every sequence of two steps and the machine arm every path of the same two steps; the Lean proves every sequence. The gap between two and every is the induction the Lean does and the C harness does not, and the GenMC clients are two threads with each loop run eight times at most. A reader who wants more coverage of the C raises `PROVE_STEPS` and pays for it in memory, which is why the movable harness does not run on a hosted runner.

In sum: the cellblock is a static memory manager whose one claim is about one word. The two boundaries of its two tiers are the two halves of that word, a take is one fetch-add that moves its own half and reads the other in the same step, a refused take leaves its bytes as an overrun for the next writer to settle, and a give is one store and one fetch-or. Lemmas 1 to 5 and Theorem 1 prove the invariant through every interleaving of a round, and Lemma 6 and the undo example prove

that the two natural alternatives, two words and an undo, break it. The layout, the take, the spill, the trim, the owner word and the movable option are theorems in Lean 4 over lists of cells and over `BitVec`; the pass and the drain take the theorems of the turnstile whole; CBMC, the machine arm and GenMC prove the C and its binaries against the model to their bounds; and 778 bytes of `-Os` Xtensa is what all of it costs on the target. Everything beyond that is an axiom, named.

BIBLIOGRAPHY

[1] ISO/IEC. *ISO/IEC 9899:2011, Programming languages, C*. International Organization for Standardization, Geneva, 2011.

[2] L. de Moura, S. Ullrich. *The Lean 4 Theorem Prover and Programming Language*. CADE 28, LNCS 12699, 625–635, 2021, doi:10.1007/978-3-030-79876-5_37.

[3] D. Quigg. *The Turnstile-Oracle Causal Ring: A decoupled synchronization and storage architecture, proved in Lean 4*. embedded_types, theory/papers/turnstile_ticket_ring, 2026.

[4] E. Clarke, D. Kroening, F. Lerda. *A Tool for Checking ANSI-C Programs*. TACAS 2004, LNCS 2988, 168–176, doi:10.1007/978-3-540-24730-2_15.

[5] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, G. Vigna. *SOK: (State of) The Art of War: Offensive Techniques in Binary Analysis*. IEEE Symposium on Security and Privacy, 138–157, 2016, doi:10.1109/SP.2016.17.

[6] M. Kokologiannakis, V. Vafeiadis. *GenMC: A Model Checker for Weak Memory Models*. CAV 2021, LNCS 12759, 427–440, 2021, doi:10.1007/978-3-030-81685-8_20.

[7] O. Lahav, V. Vafeiadis, J. Kang, C.-K. Hur, D. Dreyer. *Repairing Sequential Consistency in C/C++11*. PLDI 2017, 618–632, doi:10.1145/3062341.3062352.

[8] A. Podkopaev, O. Lahav, V. Vafeiadis. *Bridging the Gap between Programming Languages and Hardware Weak Memory Models*. POPL 2019, PACMPL 3, article 69, doi:10.1145/3290382.

[9] C. Pulte, S. Flur, W. Deacon, J. French, S. Sarkar, P. Sewell. *Simplifying ARM Concurrency: Multicopy-Atomic Axiomatic and Operational Models for ARMv8*. POPL 2018, PACMPL 2, article 19, doi:10.1145/3158107.

[10] A. Pnueli, M. Siegel, E. Singerman. *Translation Validation*. TACAS 1998, LNCS 1384, 151–166, doi:10.1007/BFb0054170.

[11] U. Drepper. *Futexes Are Tricky*. Red Hat, 2011. Also `futex(2)`, Linux Programmer's Manual.

[12] J. Evans. *A Scalable Concurrent malloc(3) Implementation for FreeBSD*. BSDCan 2006.

[13] D. Leijen, B. Zorn, L. de Moura. *Mimalloc: Free List Sharding in Action*. APLAS 2019, LNCS 11893, 244–265, doi:10.1007/978-3-030-34175-6_13.

[14] M. Masmano, I. Ripoll, A. Crespo, J. Real. *TLSF: A New Dynamic Memory Allocator for Real-Time Systems*. ECRTS 2004, 79–88, doi:10.1109/EMRTS.2004.1311009.

[15] P. R. Wilson, M. S. Johnstone, M. Neely, D. Boles. *Dynamic Storage Allocation: A Survey and Critical Review*. IWMM 1995, LNCS 986, 1–116, doi:10.1007/3-540-60368-9_19.

[16] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, S. Winwood. *seL4: Formal Verification of an OS Kernel*. SOSP 2009, 207–220, doi:10.1145/1629575.1629596.

[17] X. Leroy. *Formal verification of a realistic compiler*. Communications of the ACM 52(7), 107–115, 2009, doi:10.1145/1538788.1538814.

[18] R. Jung, R. Krebbers, J.-H. Jourdan, A. Bizjak, L. Birkedal, D. Dreyer. *Iris from the ground up: A modular foundation for higher-order concurrent separation logic*. Journal of Functional Programming 28, e20, 2018, doi:10.1017/S0956796818000151.

[19] R. Kumar, M. O. Myreen, M. Norrish, S. Owens. *CakeML: A Verified Implementation of ML*. POPL 2014, 179–191, doi:10.1145/2535838.2535841.